Лабораторная работа №1

Верификация модели турникета метро в nuXmv

Касимов Владислав Андреевич

2026

1. Постановка задачи
2. Описание системы
3. Модель в nuXmv
4. Свойства
5. Результаты верификации
6. Нетривиальность посылок
7. Анализ невыполненного свойства P4
8. Выводы
Приложение A. Модель `turnstile.smv`

1. Постановка задачи

По варианту 2 из списка CS2025 нужно смоделировать турникет в метро и верифицировать его в nuXmv. Требования к модели такие:

- в системе есть управляющий процесс, датчики, актуаторы и среда;
- среда бесконечно и недетерминированно подаёт входные сигналы;
- свойств три, и темпоральная структура у них разная;
- минимум два свойства должны выполняться. Для невыполненного свойства нужен разбор контрпримера и довод, почему исправление неприемлемо сложно;
- для свойств вида `G(p → Φ)` надо показать, что посылка `p` выполняется, причём лучше бесконечно часто.

2. Описание системы

Турникет пропускает пассажиров по транспортной карте или по жетону. Пассажир прикладывает карту или бросает жетон. Если оплата прошла, загорается зелёный сигнал и открываются створки. Пассажир проходит, фотоэлемент в проходе это фиксирует, после чего створки закрываются. Если оплата не прошла, загорается красный сигнал, пассажир забирает носитель и уходит.

Роль	**Компонент**	**Модуль**	**Состояния и выходы**
среда	пассажир	`Passenger`	`away → approach → wait → walk → away`; носитель `card_ok`, `card_empty`, `token_ok`, `token_bad`
среда	безбилетник	`Intruder`	`none → behind → sneak → none`; проскакивает вплотную за оплатившим пассажиром
датчик	валидатор карт	`CardReader`	`res ∈ {none, ok, fail}`, результат хранится, пока контроллер его не заберёт
датчик	приёмник жетонов	`TokenAcceptor`	то же для жетонов
датчик	фотоэлемент	`PassSensor`	`beam`: луч перекрыт
управление	контроллер	`Controller`	`idle`, `granted`, `denied`, `closing`
актуатор	створки	`Gate`	`closed`, `open`
индикатор	табло	`Lamp`	`off`, `green`, `red`

Цикл контроллера:

Из	**Условие**	**В**	**Действие**
`idle`	датчик выдал `ok`	`granted`	команда «открыть», зелёный

Из	Условие	В	Действие
`idle`	датчик выдал `fail`	`denied`	красный
`granted`	луч перекрыт (`beam`)	`closing`	команда «закрыть»
`denied`	всегда	`idle`	табло гаснет
`closing`	створки закрыты	`idle`	

Входные сигналы среды не ограничены. Пассажир может подойти в любой момент или не подходить вообще, носитель выбирается недетерминированно, безбилетник появляется и уходит когда угодно.

3. Модель в nuXmv

Модель синхронная. Каждый компонент описан отдельным модулем, и на каждом шаге все модули меняют состояние одновременно. Модуль видит значения других модулей на текущем шаге, поэтому реакция на событие приходит со следующего такта. Например, контроллер переходит в `granted`, на следующем такте `Gate` открывает створки, ещё через такт пассажир идёт в проход. Датчики оплаты хранят результат, пока контроллер находится не в `idle`. Так сигнал не теряется в фазе закрытия предыдущего прохода.

Полный текст модели приведён в приложении А. Ключевые фрагменты:

```
-- пассажир: приход недетерминирован
next(st) := case
  st = away                      : {away, approach};
  st = approach                  : wait;
  st = wait & gate_pos = open    : walk;
  st = wait & lamp_light = red   : away;
  st = walk                      : away;
  TRUE                           : st;
esac;

-- фотоэлемент: один бит на весь проход
next(beam) := user_st = walk | intr_st = sneak;
```

Вспомогательные определения:

```
paid     := card.res = ok   | token.res = ok;    -- датчик подтвердил оплату
rejected := card.res = fail | token.res = fail;  -- датчик отклонил оплату
```

4. Свойства

№	Логика	Формула	Структура	Смысл
P1	CTL	`AG (rejected -> A [ gate.pos = closed U lamp.light = red ])`	безопасность с Until	после отказа створки закрыты, пока не загорится красный

№	Логика	Формула	Структура	Смысл
P2	LTL	`G (paid -> F user.st = walk)`	отклик (response)	если оплата подтверждена, пассажир обязательно пройдёт
P3	LTL	`G ((gate.pos = closed & X gate.pos = open) -> X (gate.pos = open U sens.beam))`	X и Until	открывшиеся створки остаются открытыми, пока фотоэлемент не зафиксирует проход
P4	LTL	`G !(intr.st = sneak)`	инвариант	без оплаты пройти нельзя (**ожидается нарушение**)

Все четыре свойства устроены по-разному. P1 — ветвящаяся логика: Until под квантором по всем путям. P2 — классическая живость вида `G(p → F q)`. P3 использует оператор следующего шага, чтобы поймать момент открытия, и сильный Until, то есть проход обязан состояться. P4 — простой инвариант безопасности.

P1 проверяет безопасность: створки не должны открыться по отклонённой оплате. Until здесь сильный, так что заодно требуется, чтобы красный сигнал действительно загорелся.

P3 запрещает закрыть створки перед носом у пассажира. Посылка `closed ∧ X open` выделяет ровно момент открытия.

5. Результаты верификации

Прогон выполняется командой `bash lab1-nuxmv/run.sh`. Последовательность команд nuXmv лежит в `cmds.txt`: `build_model`, `print_reachable_states`, `check_fsm`, `check_ctlspec`, `check_ltlspec`.

Показатель	Значение
Достижимых состояний	**108** из 25 920 (2^6.75 из 2^14.66)
Диаметр системы	10
Тупики (`check_fsm`)	нет: «The transition relation is total»
Время всего прогона	0.024 с

Свойство	Результат
P1	**true**
P2	**true**
P3	**true**
P4	**false**, есть контрпример

Выполняются три свойства из четырёх. Невыполненное P4 разобрано в разделе 7.

6. Нетривиальность посылок

Если посылка `p` недостижима, свойство вида `G(p → Φ)` выполняется тривиально. Поэтому для каждой посылки проверено `AG EF p`: из любого достижимого состояния `p` снова достижима. Пространство состояний конечно, и в каждом состоянии есть переход, так что из `AG EF p` следует существование пути, на котором `p` выполняется бесконечно часто. Такой путь строится по шагам: из каждой точки идём к ближайшему состоянию с `p`.

Проверка	Формула	Для чего	Результат
v1	`AG EF rejected`	посылка P1	true
v2	`AG EF paid`	посылка P2	true
v3	`AG EF (gate.pos = closed & EX gate.pos = open)`	посылка P3 (момент открытия)	true
v4	`AG (EF card.res=ok & EF token.res=ok & EF card.res=fail & EF token.res=fail)`	работают оба датчика в обоих исходах	true
v5	`AG EF (ctrl.st = idle & gate.pos = closed & user.st = away)`	система всегда может вернуться в исходное состояние	true

7. Анализ невыполненного свойства P4

Контрпример nuXmv

nuXmv выдал лассо. Ниже префикс до нарушения; показаны только изменившиеся переменные:

Шаг	Изменения	Что происходит
1.1	всё в начальных значениях	турникет свободен
1.2	`user.st = approach`, `user.media = card_ok`, `intr.st = behind`	подходит пассажир с действующей картой, за ним пристраивается безбилетник
1.3	`user.st = wait`, `card.res = ok (paid)`	валидатор подтвердил оплату
1.4	`ctrl.st = granted`, `open_cmd = TRUE`	контроллер разрешает проход
1.5	`gate.pos = open`, `lamp.light = green`	створки открыты
1.6	`user.st = walk`, **`intr.st = sneak`**	пассажир входит в проход, безбилетник прямо за ним: **нарушение**
1.7	`sens.beam = TRUE`, `user.st = away`, `intr.st = none`	фотоэлемент фиксирует **один** перекрытый луч
1.8	`ctrl.st = closing`	контроллер закрывает створки, будто прошёл один человек

Дальше трасса зацикливается (с 1.7, «Loop starts here»): снова оплативший пассажир, створки снова открываются.

Причина

Фотоэлемент выдаёт один бит: «в проходе кто-то есть». Двое людей, идущих вплотную, перекрывают луч одним непрерывным интервалом. Для контроллера такой проход неотличим от прохода одного пассажира. Это не ошибка в логике контроллера: в модели просто нет датчика, который различает людей.

Почему исправление неприемлемо сложно

1. **Контроллер не может исправить это сам.** Он и так закрывает створки при первом же перекрытии луча (`granted ∧ beam → closing`), но безбилетник входит вплотную, в тот же такт, что и пассажир (шаг 1.6), то есть раньше, чем створки успевают среагировать. Закрывать ещё раньше, до перекрытия луча, нельзя: это нарушит P3 (`open U beam`) и P2, потому что створки закроются перед оплатившим пассажиром. У контроллера есть только `beam` и результат оплаты, и по ним нельзя отличить одного человека от двоих.
2. **Нужен новый класс датчиков.** Это может быть счётчик людей (несколько лучей и анализ последовательности перекрытий), датчик дистанции или механическая трипода, которая поворачивается на одну позицию за один проход. В модели это означает:
   - геометрию прохода, то есть позиции людей внутри прохода, а не одно состояние `walk`;
   - дискретное время или расстояние между людьми, иначе «вплотную» не выражается;
   - новые свойства: створки не должны ударить человека, и проход по одной оплате ровно для одного человека.

   Пространство состояний вырастет кратно по числу позиций и интервалов, а свойств придётся проверять вдвое больше.
3. **Реальные турникеты устроены так же.** Створчатые турникеты в метро тоже пропускают «паровозик». С этим борются не логикой контроллера, а видеонаблюдением и контролёрами. P4 описывает известное ограничение конструкции, а не дефект модели.

Поэтому P4 оставлено как свойство, которое модель *не обязана* выполнять. Контрпример документирует уязвимость, которую надо учитывать при эксплуатации.

8. Выводы

- В nuXmv построена синхронная модель турникета из восьми модулей: среда (2), датчики (3), управление (1), актуаторы (2). Достижимых состояний 108, тупиков нет.
- Выполняются три свойства разной темпоральной структуры: P1 (CTL, `A[·U·]`), P2 (LTL, отклик) и P3 (LTL, `X` и `U`).
- P4 нарушается из-за «паровозика». Контрпример разобран. Исправление требует нового класса датчиков и детализации модели и конфликтует с P3.
- Все посылки выполнимы бесконечно часто (`AG EF`).
- Ограничения справедливости (`FAIRNESS`) не понадобились. В синхронной композиции каждый модуль делает шаг на каждом такте, поэтому «несправедливого планировщика» просто нет. В асинхронных моделях из ЛР2 и ЛР3 это уже не так.

Приложение А. Модель `turnstile.smv`

```
-- ============================================================================
-- Лабораторная работа №1. Турникет в метро (вариант 2 из CS2025)
-- Входной язык nuXmv, синхронная композиция модулей.
--
-- Компоненты (каждый модуль = отдельный «процесс» системы):
--   Passenger     - пользователь/среда: время от времени подходит и
--                   недетерминированно предъявляет карту или жетон
--   Intruder      - среда: «безбилетник», пытается проскочить следом
--   CardReader    - датчик: валидатор карт (есть деньги / нет денег)
--   TokenAcceptor - датчик: приёмник жетонов (правильный / посторонний)
--   PassSensor    - датчик: фотоэлемент в проходе
--   Controller    - управляющий алгоритм
--   Gate          - актуатор: створки
--   Lamp          - актуатор/индикатор: табло (зелёный / красный)
--
-- Каждый модуль реагирует на значения остальных с задержкой в 1 такт.
-- ============================================================================

MODULE Passenger(gate_pos, lamp_light)
VAR
  st    : {away, approach, wait, walk};
  media : {none, card_ok, card_empty, token_ok, token_bad};
ASSIGN
  init(st)    := away;
  init(media) := none;
  next(st) := case
    st = away                      : {away, approach}; -- приходит когда угодно
    st = approach                  : wait;             -- приложил карту / бросил жетон
    st = wait & gate_pos = open    : walk;             -- створки открыты - идёт
    st = wait & lamp_light = red   : away;             -- отказ - уходит
    st = walk                      : away;             -- прошёл
    TRUE                           : st;
  esac;
  next(media) := case
    st = away & next(st) = approach : {card_ok, card_empty, token_ok, token_bad};
    next(st) = away                 : none;
    TRUE                            : media;
  esac;

MODULE Intruder(gate_pos)
VAR
  st : {none, behind, sneak};
ASSIGN
  init(st) := none;
  next(st) := case
    st = none                      : {none, behind};
    st = behind & gate_pos = open : {behind, sneak, none}; -- может проскочить
    st = behind                    : {behind, none};
    st = sneak                     : none;
    TRUE                           : st;
  esac;

-- Датчик держит результат, пока контроллер его не заберёт (латч)
MODULE CardReader(user_st, user_media, ctrl_st)
VAR
  res : {none, ok, fail};
```

```
ASSIGN
  init(res) := none;
  next(res) := case
    res != none & ctrl_st = idle                              : none;
    res = none & user_st = approach & user_media = card_ok     : ok;
    res = none & user_st = approach & user_media = card_empty  : fail;
    TRUE                                                       : res;
  esac;

MODULE TokenAcceptor(user_st, user_media, ctrl_st)
VAR
  res : {none, ok, fail};
ASSIGN
  init(res) := none;
  next(res) := case
    res != none & ctrl_st = idle                              : none;
    res = none & user_st = approach & user_media = token_ok    : ok;
    res = none & user_st = approach & user_media = token_bad   : fail;
    TRUE                                                       : res;
  esac;

-- Фотоэлемент: луч перекрыт, если в проходе кто-то есть
MODULE PassSensor(user_st, intr_st)
VAR
  beam : boolean;
ASSIGN
  init(beam) := FALSE;
  next(beam) := user_st = walk | intr_st = sneak;

MODULE Controller(card_res, token_res, beam, gate_pos)
VAR
  st : {idle, granted, denied, closing};
ASSIGN
  init(st) := idle;
  next(st) := case
    st = idle & (card_res = ok   | token_res = ok)   : granted;
    st = idle & (card_res = fail | token_res = fail) : denied;
    st = granted & beam                              : closing;
    st = denied                                      : idle;
    st = closing & gate_pos = closed                 : idle;
    TRUE                                             : st;
  esac;
DEFINE
  open_cmd := st = granted;

MODULE Gate(open_cmd)
VAR
  pos : {closed, open};
ASSIGN
  init(pos) := closed;
  next(pos) := open_cmd ? open : closed;

MODULE Lamp(ctrl_st)
VAR
  light : {off, green, red};
ASSIGN
  init(light) := off;
  next(light) := case
    ctrl_st = granted : green;
    ctrl_st = denied  : red;
    TRUE              : off;
```

```
  esac;

MODULE main
VAR
  user  : Passenger(gate.pos, lamp.light);
  intr  : Intruder(gate.pos);
  card  : CardReader(user.st, user.media, ctrl.st);
  token : TokenAcceptor(user.st, user.media, ctrl.st);
  sens  : PassSensor(user.st, intr.st);
  ctrl  : Controller(card.res, token.res, sens.beam, gate.pos);
  gate  : Gate(ctrl.open_cmd);
  lamp  : Lamp(ctrl.st);
DEFINE
  paid     := card.res = ok   | token.res = ok;    -- датчик подтвердил оплату
  rejected := card.res = fail | token.res = fail;  -- датчик отклонил оплату

-- ---------------------------------------------------------------------
-- P1 (CTL, безопасность с Until): если оплата отклонена, створки
--    остаются закрытыми, пока не загорится красный сигнал.
CTLSPEC NAME p1_reject_keeps_closed :=
  AG (rejected -> A [ gate.pos = closed U lamp.light = red ])

-- P2 (LTL, отклик G(p -> F q)): пассажир, чья оплата подтверждена,
--    обязательно пройдёт через турникет.
LTLSPEC NAME p2_paid_passes :=
  G (paid -> F user.st = walk)

-- P3 (LTL, X и U): открывшиеся створки остаются открытыми, пока
--    датчик прохода не зафиксирует проход.
LTLSPEC NAME p3_open_until_pass :=
  G ((gate.pos = closed & X gate.pos = open) -> X (gate.pos = open U sens.beam))

-- P4 (LTL, инвариант; ОЖИДАЕТСЯ НАРУШЕНИЕ): без оплаты пройти нельзя.
--    Контрпример - «паровозик» (tailgating), разбор в отчёте.
LTLSPEC NAME p4_no_free_ride :=
  G !(intr.st = sneak)

-- ---------------------------------------------------------------------
-- Проверки нетривиальности посылок: из любого достижимого состояния
-- посылка снова достижима => она может выполняться бесконечно часто.
CTLSPEC NAME v1_rejected_recurs := AG EF rejected
CTLSPEC NAME v2_paid_recurs     := AG EF paid
CTLSPEC NAME v3_opening_recurs  := AG EF (gate.pos = closed & EX gate.pos = open)
CTLSPEC NAME v4_card_and_token  := AG (EF card.res = ok & EF token.res = ok
                                       & EF card.res = fail & EF token.res = fail)
-- Система не «зависает»: в исходное состояние можно вернуться всегда
CTLSPEC NAME v5_can_return_idle := AG EF (ctrl.st = idle & gate.pos = closed & user.st =
away)
```